# RIO: A Runtime Governance and Execution Control Protocol for AI Systems

**Version 2.0 — March 2026 Author: Brian Kent Rasmussen Repository: github.com/bkr1297-RIO/rio-system**

---

## Abstract

As AI agents gain the ability to execute real-world actions — sending emails, committing code, transferring funds, modifying infrastructure — the absence of a structural enforcement layer between the agent and the target system creates a class of risk that cannot be addressed by prompt engineering, model alignment, or client-side confirmation dialogs. RIO (Runtime Intelligence Orchestration) is a governed execution protocol that interposes a fail-closed control plane between AI systems and external APIs. Every action must traverse a deterministic pipeline — Intent, Governance, Execution, Receipt — before reaching the target system. The protocol enforces server-side authorization, generates cryptographically signed receipts for every action, and appends those receipts to a tamper-evident hash-chained ledger. This paper describes the protocol specification, the threat model, the reference implementations, and the verification results from two independent codebases totaling over 327 automated tests.

---

## 1. Problem Statement

The current generation of AI agent frameworks provides mechanisms for AI systems to interact with external services through tool-use APIs, OAuth connectors, and function-calling interfaces. These frameworks give AI agents the *ability* to act but provide no structural *governance* over those actions. The connection between an AI agent and an

external API is a direct pipe with no intermediary enforcing authorization, logging, or verification.

Existing approaches to controlling AI agent behavior fall into three categories, all of which are insufficient for production environments:

| Approach | Mechanism | Failure Mode |
|---|---|---|
| Prompt Engineering | System instructions directing the AI to “be careful” | Advisory only; overridden by prompt injection or model drift |
| Model Alignment | Training-time reinforcement of desired behaviors | Probabilistic; no runtime guarantee of compliance |
| UI Confirmation | Frontend dialog asking user to click “Confirm” | Client-side only; removable by any party with platform access |

None of these approaches provide structural enforcement. They rely on the AI’s willingness to follow rules, the stability of trained behaviors, or the integrity of a client-side interface. In a production environment where an AI agent holds OAuth tokens to Gmail, GitHub, or a payment processor, the absence of a server-side execution gate means that a single prompt injection, a platform configuration change, or a model update can result in unauthorized actions with no audit trail.

The core problem is architectural: the tools that give AI agents the ability to act in the real world have no built-in governance layer. RIO addresses this gap.

---

## 2. Solution: Governed AI Execution

RIO is not an AI model, not a chatbot framework, and not a monitoring dashboard. It is a control plane through which every AI-initiated action must pass before reaching the target system. The runtime enforces three guarantees:

**No execution without authorization.** The system defaults to blocking all actions. Execution proceeds only after the governance pipeline produces a valid, signed approval. This is a fail-closed architecture: if any component is unavailable, missing, or invalid, execution is denied.

**Every action produces a cryptographic receipt.** Each completed action generates a receipt containing hashes of the original intent, the governance decision, the authorization record, the executed action, and the verified outcome. Receipts are signed and appended to a hash-chained ledger, creating a tamper-evident audit trail.

**Governance is structural, not advisory.** Authorization is enforced server-side at the execution gateway. The AI agent cannot bypass the gate by modifying its own instructions, exploiting a prompt injection, or accessing tools directly. The gateway holds the OAuth tokens and API credentials; the agent does not.

---

# 3. Core Loop: Intent, Governance, Execution, Receipt

RIO operates on a single deterministic pipeline that every action must traverse. The pipeline consists of four stages, each producing an immutable artifact that is hashed and chained to the next.

## 3.1 Intent

The AI agent or an upstream system submits a structured intent describing the proposed action. The intent includes the action type (e.g., `send_email`, `create_pr`, `transfer_funds`), the target system, the parameters, and the requesting identity. Once submitted, the intent is immutable and assigned a unique identifier. The intent is hashed using SHA-256 over its canonical JSON representation.

## 3.2 Governance

The governance stage evaluates the intent against the active policy set. The policy engine classifies the action by risk level and returns one of three verdicts:

| Verdict | Condition | Effect |
|---|---|---|
| `AUTO_APPROVE` | Action falls within pre-authorized boundaries (low-risk, routine) | Execution proceeds without human intervention |
| `REQUIRE_HUMAN` | Action exceeds autonomous thresholds or has external effect | Execution blocked until human authority approves |
| `AUTO_DENY` | Action violates a hard policy constraint | Execution permanently blocked |

For actions requiring human approval, the system presents the intent to the designated authority through the Gate UI (web interface, Slack interactive message, or mobile notification). The authority reviews the proposed action, the risk assessment, and the policy evaluation, then issues an approval or denial. Approvals are cryptographically signed, time-limited (default TTL: 300 seconds), and bound to the specific intent identifier.

The governance decision itself is hashed and recorded as part of the receipt chain.

## 3.3 Execution

The execution gateway receives the approved intent and performs four verification checks before allowing the action to proceed:

1. A valid approval record exists and its cryptographic signature is verified.
2. The approval has not expired.
3. The approval has not been previously consumed (single-use enforcement).
4. The execution parameters match the original intent exactly (no parameter tampering).

If all checks pass, the gateway dispatches the action to the appropriate connector (Gmail, Google Drive, GitHub, Slack, Outlook, OneDrive, Google Calendar) and records the execution result. If any check fails, execution is denied and a denial receipt is generated.

The execution record is hashed and becomes part of the receipt chain.

## 3.4 Receipt

Immediately following execution, the system generates a cryptographic receipt. The receipt binds together five hashed artifacts into a single signed document:

| Artifact | Content | Hash Field |
|---|---|---|
| Intent | Original action request | `intent_hash` |
| Governance | Policy evaluation and risk assessment | `governance_hash` |
| Authorization | Human approval or denial record | `authorization_hash` |
| Execution | Action result from target system | `execution_hash` |
| Receipt | Final binding document | `receipt_hash` |

The receipt hash is computed over the concatenation of the four preceding hashes, the receipt identifier, and the timestamp. The receipt is then signed using Ed25519 (demo site) or ECDSA secp256k1 (gateway), and appended to the ledger with a `previous_hash` field pointing to the preceding receipt's hash, forming a tamper-evident chain.

---

# 4. System Architecture

The RIO system is composed of four layers, each with a distinct responsibility and a clear trust boundary.

## 4.1 Architectural Layers

**Layer 1: Agent / Planner.** The AI system (any LLM or agent framework) that proposes actions. This layer has no direct access to external APIs. It can only submit intents to the governance layer. The agent is untrusted — the system assumes it may be compromised by prompt injection, model drift, or adversarial input.

**Layer 2: Governance Gateway.** The server-side control plane that receives intents, evaluates policy, manages approvals, and gates execution. This is the trust boundary. The gateway holds the policy configuration, the cryptographic keys, and the connector credentials. It exposes a defined API surface and enforces fail-closed behavior on every endpoint.

**Layer 3: Connectors.** Typed adapters that translate authorized intents into API calls against external systems. Each connector (Gmail, GitHub, Google Drive, Slack,

Outlook, OneDrive, Google Calendar) implements a standard interface with `execute(intent, receipt)` and `simulate(intent)` methods. Connectors operate in two modes: `simulated` (returns a mock result for testing) and `live` (executes the real API call). The connector registry routes actions to the correct connector based on action type.

**Layer 4: Ledger.** The append-only, hash-chained storage for receipts. Each entry contains the receipt, its `ledger_hash`, and a `previous_hash` linking it to the preceding entry. The ledger supports integrity verification by recomputing the hash chain from genesis.

## 4.2 Endpoint Surface

The governance gateway exposes the following endpoints:

| Endpoint | Method | Purpose |
|---|---|---|
| `/intent` | POST | Submit a new intent from any AI agent |
| `/govern` | POST | Run policy and risk evaluation on an intent |
| `/authorize` | POST | Record human approval or denial |
| `/execute` | POST | Execute an authorized action through a connector |
| `/receipt` | POST | Generate cryptographic receipt |
| `/ledger` | GET | View ledger entries |
| `/verify` | GET | Verify receipt hash chain integrity |
| `/health` | GET | System health check |
| `/intents` | GET | List all intents |
| `/intent/:id` | GET | Get a specific intent with full pipeline state |

All endpoints enforce fail-closed behavior: if any prerequisite is missing, the request returns an error and execution does not proceed.

---

# 5. Receipt Protocol

The RIO Receipt Protocol defines five data structures that form the complete chain of custody for a governed action. The protocol is specified as a JSON Schema (version 1.0.0) and is published in the `rio-receipt-protocol/schemas/` directory of the reference repository.

## 5.1 Data Structures

**ActionRequest.** An immutable record of a proposed action. Required fields: `request_id`, `action_type`, `requested_by`, `target`, `parameters`, `risk_level`, `created_at`. The action type uses a verb-noun convention (e.g., `send_email`, `create_pr`, `transfer_funds`). Risk levels are enumerated as `low`, `medium`, `high`, `critical`.

**AIRecommendation.** The AI system's evaluation of the request against policy. Required fields: `recommendation_id`, `request_id`, `recommended_action` (`proceed` | `block` | `escalate`), `confidence` (0.0 to 1.0), `reasoning`, `created_at`. The recommendation may include an `unknowns` array declaring assumptions that affect confidence.

**ApprovalRecord.** A cryptographically signed authorization from a human authority. Required fields: `approval_id`, `request_id`, `approver_id`, `approval_status` (`approved` | `denied` | `revoked` | `expired`), `approved_at`, `auth_method`, `signature`, `signature_algorithm`. The approval includes `expires_at` for time-limited authorization and `conditions` for scoped approval.

**ExecutionRecord.** The record of what was actually executed. Required fields: `execution_id`, `request_id`, `approval_id`, `executed_by`, `executed_at`, `execution_status` (`success` | `failed` | `partial` | `rolled_back`), `execution_token_id`. The execution token is a one-time token consumed during execution, proving authorization and preventing replay.

**Receipt.** The final, immutable record binding all four preceding artifacts. Required fields: `receipt_id`, `request_id`, `recommendation_id`, `approval_id`, `execution_id`, `action_type`, `requested_by`, `approver_id`, `executed_by`, `created_at`, `ledger_hash`, `previous_hash`, `signature`, `verification_method`,

`status`. The `previous_hash` field contains the `ledger_hash` of the preceding receipt (or `GENESIS` for the first receipt), creating the tamper-evident chain.

### 5.2 Hash Specification

All hashes use SHA-256 over deterministic JSON serialization (keys sorted alphabetically, no whitespace, UTF-8 encoding). The genesis value for the first receipt in a chain is the string `GENESIS`.

### 5.3 Supported Signature Methods

The protocol supports four signature methods: `hmac_sha256`, `ecdsa_secp256k1`, `rsa_pss_sha256`, and `ed25519`. The reference implementation uses Ed25519 for the demo site and ECDSA secp256k1 for the standalone gateway.

---

## 6. Security Model

The security model is grounded in four principles that apply uniformly across all threat categories.

### 6.1 Design Principles

**Fail-Closed by Default.** The system's default state is "execution blocked." Every component must explicitly authorize its part of the chain. If any component is missing, unavailable, or returns an unexpected result, execution does not proceed. This applies to the approval service, the ledger service, the signature service, and the connector layer.

**Server-Side Enforcement.** All authorization checks are enforced on the backend server. The frontend UI is a convenience layer for human interaction — it is not a security boundary. An attacker who bypasses the frontend still faces the same server-side enforcement.

**Cryptographic Binding.** Approvals are not database flags. They are cryptographically signed artifacts bound to a specific payload, a specific intent, and a specific moment in time. Signatures are verified independently at execution time.

**Tamper-Evident Recording.** Every decision and action is recorded in a chained-hash ledger. The ledger does not prevent tampering — it makes tampering detectable. Any modification to any record breaks the hash chain and is visible during audit.

## 6.2 Threat Model

The threat model assumes that every component outside the server-side control plane is potentially compromised — including the AI agent, the network, and the frontend. The following threats are addressed:

| Threat ID | Threat | Risk | Mitigation | Verification |
|---|---|---|---|---|
| T-001 | Direct executor call (bypass gateway) | Critical | Service boundary + service-to-service auth | V-006: PASS |
| T-002 | Execution without approval | Critical | Fail-closed gateway, HTTP 403, denial receipt | V-001: PASS |
| T-003 | Forged signature | Critical | Cryptographic verification, key isolation | V-007: PASS |
| T-004 | Fabricated approval record | High | Signature verification + chained ledger | V-007: PASS |
| T-005 | Replay (reuse consumed approval) | High | Single-use approvals, consumed tracking | V-003: PASS |
| T-006 | Duplicate execution | High | Idempotency enforcement | V-010: PASS |
| T-007 | Payload tampering after approval | Critical | Signature bound to payload hash | V-004: PASS |
| T-008 | Revocation race condition | High | Execution-time revocation check | V-005: PASS |
| T-009 | Ledger tampering | High | Chained hash, append-only, integrity checks | Ledger audit |
| T-010 | Ledger service unavailable | Critical | Fail-closed (no record, no execution) | V-008: PASS |
| T-011 | Approval service unavailable | Critical | Fail-closed (no approval, no execution) | V-009: PASS |
| T-012 | Signature service unavailable | High | Fail-closed (no signature, no execution) | Fail-closed pattern |

## 6.3 Three-Layer Authorization Gate

The execution gateway enforces a three-layer gate before any action reaches an external system:

**Layer 1: Nonce Registry.** The SHA-256 hash of the approval signature is checked against the `used_signatures` store. If the hash exists, the request is rejected with HTTP 409. This prevents replay attacks and enforces single-use approvals.

**Layer 2: Signature Verification.** The intent text and timestamp are verified against the public key. Any tampering to the intent invalidates the signature. Timestamps must be within a 300-second freshness window.

**Layer 3: Execution Token Verification.** A separate execution token, cryptographically bound to the full parameter set (intent, source, signature, timestamp, nonce), must be presented. This ensures the execution gate has explicitly authorized this specific request.

All three layers must pass. Failure at any layer blocks execution. All layers are fail-closed — any error (database unavailable, key missing, verification exception) results in a block, not a pass.

---

# 7. Current Implementation

The RIO protocol has two independent implementations, both open-source under the `bkr1297-RIO/rio-system` repository.

## 7.1 Demo Site (TypeScript / React / tRPC)

The demo site is a full-stack web application deployed at `riodemo-ux2sxdqo.manus.space`. It provides an interactive demonstration of the complete governance pipeline with a real backend enforcement engine.

| Component | Technology |
| --- | --- |
| Frontend | React 19, Tailwind CSS 4, shadcn/ui |
| Backend | Express 4, tRPC 11 |
| Database | TiDB (MySQL-compatible) via Drizzle ORM |
| Cryptography | Ed25519 (Node.js `crypto` module) |
| Authentication | OAuth (Manus, Google, Microsoft, GitHub) |
| Notifications | Slack interactive messages, web push |

The demo site implements the full pipeline: intent creation, policy evaluation, human approval (via web UI or Slack), execution gating, receipt generation, and ledger verification. The `approve` and `deny` endpoints use `protectedProcedure` to bind the approver's identity from the authenticated session (`ctx.user`), not from client-supplied input.

The implementation includes a connector registry with seven typed connectors (Gmail, Google Calendar, Google Drive, GitHub, Slack, Outlook Mail, Outlook Calendar) that can operate in simulated or live mode. The GitHub connector supports per-user OAuth tokens for creating issues, pull requests, and file commits through the user's own GitHub identity.

The demo site includes a policy engine that supports persistent governance rules with `AUTO_APPROVE`, `AUTO_DENY`, and `REQUIRE_HUMAN` verdicts based on action type, risk level, and custom conditions.

**Test coverage:** 327 tests across 21 test files, all passing. Tests cover intent lifecycle, approval flows, denial flows, execution gating, receipt verification, ledger chain integrity, policy persistence, connector routing, Slack interactive approvals, identity binding, and OAuth state management.

## 7.2 Standalone Gateway (Node.js / Express)

The standalone gateway is a pure API server designed for deployment as an independent service. It implements the same governance pipeline without a frontend, exposing 10 REST endpoints.

| Component | Technology |
| --- | --- |
| Server | Express, Node.js |
| Ledger | JSON file (append-only) |
| Policy | JSON configuration files (Constitution + Policy) |
| Cryptography | SHA-256 hash chain |

The gateway loads a governance constitution (`RIO_CONSTITUTION.json`) and a policy file (`RIO_POLICY.json`) at startup. The constitution defines immutable invariants (identity, consent, persistence, confidence). The policy defines agent permissions, restricted actions, approval requirements, and runtime rules.

The policy distinguishes between **build authority** (granted by default — design, draft, write code, configure, prepare) and **deploy authority** (withheld pending explicit human approval — deploy to production, send external communications, modify live security controls, execute irreversible external actions).

## 7.3 Receipt Protocol Schemas

The `rio-receipt-protocol/` directory contains a standalone, implementation-independent specification of the receipt protocol:

| File | Content |
|---|---|
| `schemas/action-request.schema.json` | ActionRequest JSON Schema |
| `schemas/ai-recommendation.schema.json` | AIRecommendation JSON Schema |
| `schemas/approval-record.schema.json` | ApprovalRecord JSON Schema |
| `schemas/execution-record.schema.json` | ExecutionRecord JSON Schema |
| `schemas/receipt.schema.json` | Receipt JSON Schema |
| `docs/protocol_spec.md` | Protocol specification |
| `docs/ledger_spec.md` | Ledger specification |
| `docs/glossary.md` | Terminology definitions |
| `python/sign_receipt.py` | Python receipt signing utility |
| `python/verify_receipt.py` | Python receipt verification utility |
| `examples/full-chain-example.json` | Complete chain example |

---

# 8. Governance Configuration

The RIO governance model is defined by two configuration documents: the Constitution and the Policy.

## 8.1 Constitution

The Constitution defines immutable invariants that cannot be overridden by policy:

> ***Identity:*** *Every action must have a known origin (agent_id) and a known target (human authority).*
>
> ***Consent:*** *No action with external effect may execute without explicit human authorization.*

> ***Persistence:*** *Every authorized action must produce a receipt, and every receipt must be written to the ledger.*
>
> ***Confidence:*** *Every AI recommendation must carry a confidence score. Actions below threshold require human escalation.*

The Constitution also defines the governance pipeline order (intent, governance, risk evaluation, authorization, execution, receipt, ledger, verification) and the fail mode (closed).

### 8.2 Policy

The Policy defines the operational rules within the constitutional framework. It specifies:

- **Scope:** Which systems, agents, and environments are governed.
- **Agent Permissions:** Actions allowed by default (design, draft, write code, configure) versus actions restricted until approved (deploy, publish, send external communications, modify live security).
- **Approval Requirements:** Required fields for approval requests (change summary, target environment, expected effects, risks, rollback plan, expiration).
- **Runtime Rules:** Behavioral constraints (prepare to the edge of deployment by default; pause before restricted actions; if scope is ambiguous, do not proceed).

---

## 9. Deployment Architecture

The target deployment architecture consists of three layers designed for $^{24}/_{7}$ operation.

### 9.1 Components

| Component | Current State | Target State |
|---|---|---|
| Frontend (Demo + Approval UI) | Manus-hosted ( `manus.space` ) | Azure App Service or equivalent |
| Gateway (Backend API) | In-memory + TiDB | Azure App Service + PostgreSQL |
| Ledger (Database) | TiDB / JSON file | Azure Database for PostgreSQL (append-only) |
| Authentication | Manus OAuth + Google + Microsoft + GitHub | Azure Entra ID (OAuth) |
| Code Source | GitHub ( `bkr1297-RIO/rio-system` ) | GitHub (Continuous Deployment) |
| Domain | `riodemo-ux2sxdqo.manus.space` | Custom domain (e.g., `rioprotocol.com` ) |

### 9.2 Security Requirements

The permanent deployment must maintain the strict security guarantees of the protocol: fail-closed execution (HTTP 403 if authorization is missing, invalid, or expired), single-use approvals (nonce registry prevents replay), tamper-evident ledger (every receipt includes `previous_hash` ), and key isolation (private keys stored in Azure Key Vault or equivalent, never uploaded to the application server).

---

## 10. Use Cases

### 10.1 AI Agent Email Governance

An AI agent proposes to send an email on behalf of a user. The intent is submitted to RIO, which evaluates the action against policy. Because `send_email` has external effect, the policy engine returns `REQUIRE_HUMAN` . The user receives a Slack notification with the email subject, recipient, and body. The user approves. The gateway verifies

the approval signature, dispatches the email through the Gmail connector, and generates a receipt proving who authorized the email, what was sent, and when.

### 10.2 Code Deployment Governance

An AI agent prepares a pull request on GitHub. Under the governance policy, `create_pr` is classified as a build action (allowed by default). The agent creates the PR without human intervention, and a receipt is generated. However, when the agent subsequently proposes `deploy_production`, the policy engine classifies this as a restricted action requiring explicit human approval. The deployment is blocked until the human authority reviews and approves.

### 10.3 Financial Transaction Governance

An AI agent proposes a fund transfer. The action is classified as `critical` risk. The policy engine requires human approval with enhanced verification (MFA or biometric). The approval is time-limited to 60 seconds and bound to the exact transfer parameters. Any modification to the amount, recipient, or account after approval invalidates the signature and blocks execution.

### 10.4 Multi-Agent Orchestration

Multiple AI agents (planning agent, execution agent, verification agent) operate within a single RIO-governed environment. Each agent submits intents through the same gateway. The policy engine applies agent-specific permissions — the planning agent can draft and recommend, but only the execution agent (with human approval) can perform external actions. All agents' actions are recorded in the same ledger, providing a unified audit trail across the entire agent system.

---

# 11. Definitions

| Term | Definition |
|---|---|
| **Intent** | A structured, immutable proposal for an action, submitted by an AI agent or upstream system. |
| **Governance** | The process of evaluating an intent against policy, constitution, and risk thresholds. |
| **Approval** | A cryptographically signed, time-limited, single-use authorization from a human authority. |
| **Execution** | The controlled dispatch of an authorized action to a target system through a connector. |
| **Receipt** | A cryptographically signed, hash-chained record binding intent, governance, approval, and execution. |
| **Ledger** | An append-only, tamper-evident store of receipts forming a hash chain from genesis. |
| **Fail-Closed** | A design principle where the default system state is “execution blocked.” |
| **Connector** | A typed adapter that translates authorized intents into API calls against external systems. |
| **Constitution** | The immutable invariants that define the governance framework. |
| **Policy** | The operational rules within the constitutional framework that govern agent permissions. |
| **Non-Repudiation** | The property ensuring no party can deny involvement in a governed action. |
| **Provenance** | The complete, traceable history of a decision from intent through execution. |

## 12. Conclusion

RIO addresses a structural gap in the current AI agent ecosystem: the absence of a server-side governance layer between AI systems and the external APIs they interact with. The protocol provides fail-closed execution gating, cryptographic receipt generation, and tamper-evident ledger recording. The system enforces the rules — not the AI.

Two independent implementations demonstrate the protocol's viability: a full-stack demo site with 327 passing tests covering the complete governance pipeline, and a standalone gateway designed for deployment as an independent service. The receipt protocol is specified as a set of JSON Schemas with reference implementations in both TypeScript and Python.

The protocol is designed to be model-agnostic, platform-agnostic, and connector-extensible. Any AI system that can submit a structured intent can be governed by RIO. Any external API that can be wrapped in a connector can be protected by the execution gate. The governance configuration (constitution and policy) is declarative and separable from the implementation, allowing organizations to define their own invariants and approval workflows.

The source code, schemas, threat model, and verification results are available at `github.com/bkr1297-RIO/rio-system`.

---

## References


[1] RIO System Repository. GitHub. https://github.com/bkr1297-RIO/rio-system

[2] RIO Demo Site. https://riodemo-ux2sxdqo.manus.space

[3] RIO Receipt Protocol Specification. `rio-receipt-protocol/docs/protocol_spec.md` in [1].

[4] RIO Threat Model. `THREAT_MODEL.md` in [1].

[5] RIO Verification Results. `VERIFICATION_RESULTS.md` in [1].

[6] RIO Governance Constitution. `gateway/config/rio/RIO_CONSTITUTION.json` in [1].

[7] RIO Governance Policy. `gateway/config/rio/RIO_POLICY.json` in [1].